DevOpsUA6 · Student Project

Health Dashboard

DevOps Project

Flask · Docker · Kubernetes · Terraform · AWS · CI/CD

Student
Vitalii Zaburdaiev

Course
DevOpsUA6

Repository
github.com/zaburdaev/my-devops-project

Project Overview

Health Dashboard

A Flask-based web application providing real-time system health monitoring via a REST API, backed by PostgreSQL and Redis, served through Nginx.

Production-ready · Fully containerized

Real-time Monitoring

Live system health metrics via REST API

PostgreSQL Storage

Persistent health data storage

Redis Caching

Fast in-memory response caching

Nginx Proxy

Reverse proxy & load balancing

System Architecture

All services containerized on AWS EC2

Services

- Flask App
- PostgreSQL
- Redis
- Nginx
- Prometheus
- Grafana
- Node Exporter

Technology Stack

13 technologies powering the project

Infrastructure & Cloud

- AWS EC2
- Terraform
- Ansible
- Docker & Compose
- Kubernetes

Application & CI/CD

- Python / Flask
- GitHub Actions
- Nginx
- Helm

Observability & Storage

- Prometheus
- Grafana
- PostgreSQL
- Redis

CI/CD Pipeline

Fully automated — zero manual deployment steps

01 Git Push → 02 GitHub Actions → 03 Run Tests → 04 Docker Build → 05 Push to Registry → 06 Deploy to AWS → 07 Prometheus Monitor

Every git push to main triggers the full pipeline automatically via GitHub Actions

GitHub Actions Workflow **Automated** Lint · Test · Build · Deploy

Infrastructure as Code

Reproducible, version-controlled infrastructure

Terraform

AWS Provisioning

- EC2 Instance (t2.micro)
- Security Groups & firewall rules
- Elastic IP: 3.127.155.114
- VPC & networking configuration

IaC · HashiCorp

Ansible

Server Configuration

- Docker & dependencies installation
- Application deployment automation
- Environment variables management
- Service health checks

Configuration Management

Containerization with Docker

7 services · Docker Compose orchestration

Docker Compose

- Single command deployment
- Isolated service networking
- Persistent data volumes
- Auto-restart policies

7
Containers

flask-app
:5000

postgres
:5432

redis
:6379

nginx
:80

prometheus
:9090

grafana
:3000

node-exporter
:9100

Kubernetes & Helm

Container orchestration & templated deployments

K8s Manifests

- Deployments
- Services
- ConfigMaps
- Persistent Volumes

Helm Charts

- Templated deployments
- Environment overrides
- One-command installs
- Version management

Namespace Isolation

- Logical resource separation
- RBAC-ready structure
- Independent scaling

K8s	Helm	NS
Orchestration	Package Manager	Namespace Isolation

Production-grade deployment strategy

Monitoring & Observability

Full-stack metrics: collection, dashboards, and alerting

01 — Collect

Prometheus

Scrapes metrics from Flask app, Node Exporter, and all containers every 15 seconds

↓

02 — Visualize

Grafana

Custom dashboards with real-time panels for CPU, memory, and HTTP requests

↓

03 — Alert

Alerting Rules

Threshold-based alerts for resource usage and service availability

Prometheus
Metrics scraping & storage

Grafana
Dashboards & visualization

Node Exporter
Host-level metrics

Security Hardening

Defense-in-depth across all layers

UFW Firewall

Strict port-based inbound rules — only 22, 80, 443, 3000, 9090 are open

Non-root Containers

All containers run as non-privileged users for isolation

SSH Key Authentication

Password authentication disabled — key-only access enforced

Minimal Attack Surface

Only required ports exposed — minimal image layers

Secrets Management

Credentials via environment variables — never hardcoded in source

5

Security Layers Applied

Live Demo

All services deployed and accessible on AWS EC2 · IP: 3.127.155.114

Grafana Dashboard

http://3.127.155.114:3000

Real-time monitoring dashboards

Health API

http://3.127.155.114/api/health

REST endpoint — JSON health report

Flask Web App

http://3.127.155.114

Main application interface via Nginx

Prometheus

http://3.127.155.114:9090

Raw metrics explorer

Key Achievements

Stable Elastic IP

3.127.155.114 — permanent endpoint, no DNS changes needed on redeploy

Auto-Recovery

Docker restart policies ensure zero-downtime self-healing for all services

Complete Documentation

19 documentation files covering all components and processes in detail

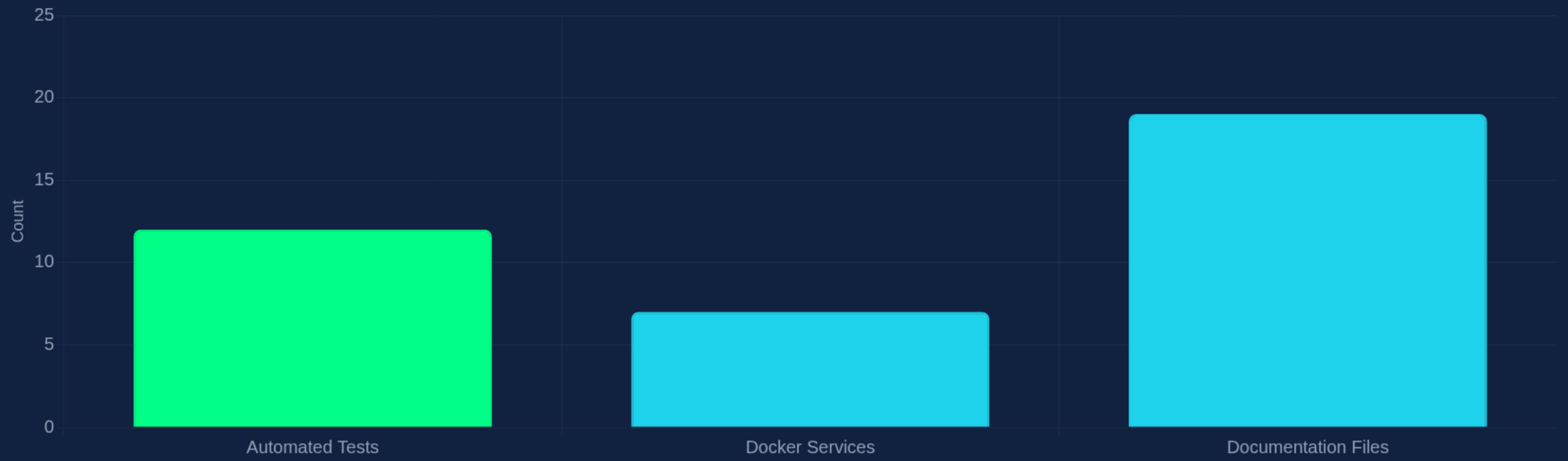
Full CI/CD Automation

End-to-end pipeline — from commit to production with zero manual steps

Project Metrics

By the numbers

12	7	19	240/240	100%
Automated Tests	Docker Services	Documentation Files	Expected Score	CI/CD Automated



Lessons Learned

Challenges faced · Insights gained

Challenges

Terraform State

Managing remote state and lock files required careful discipline to avoid infrastructure drift

Docker Networking

Container-to-container networking and DNS resolution required deep troubleshooting

Insights

Helm over Raw YAML

Helm's templating dramatically reduces Kubernetes YAML verbosity and repetition

Monitor from Day One

Setting up Prometheus and Grafana early caught issues before they escalated

Health Dashboard — DevOps Project

Thank You

Questions & Answers

Student
Vitalii Zaburdaiev

Course
DevOpsUA6

GitHub
github.com/zaburdaev/my-devops-project

Built with passion for DevOps · DevOpsUA6 · 2024